

QuickDelivery - App Flutter

1. Objetivo

O app Flutter unificado implementa a entrega da Sprint 3 para o cliente e já prepara a Sprint 4 para o entregador. Ele permite que um usuário com papel `CUSTOMER` faça login, consulte suas entregas, crie novas solicitações, acompanhe detalhes e cancele entregas quando a máquina de estados permitir. Usuários com papel `DELIVERYMAN` também conseguem autenticar e acessar uma tela inicial simples de entregador, deixando o fluxo operacional completo do prestador para a Sprint 4.

2. Telas

Login

A tela de login consome `POST /auth/login`, mantém o token em memória durante a sessão e direciona o usuário conforme o papel retornado pelo backend.

Minhas Entregas

A tela principal consome `GET /deliveries` e lista apenas as entregas do cliente autenticado, conforme a autorização do backend. Cada card mostra descrição, origem, destino, status e data de atualização. A tela possui ação manual de atualizar, acesso ao perfil e botão para criar nova entrega.

Nova Entrega

A tela de criação envia `POST /deliveries` com `customerId`, `pickupAddress`, `dropoffAddress` e `description`. Após criação bem-sucedida, o app abre o detalhe da entrega criada.

Detalhes da Entrega

A tela de detalhe consome `GET /deliveries/:id`, exibe status, origem, destino, descrição, entregador atribuído e datas. O cliente pode cancelar a entrega quando o status está em `PENDING` ou `ACCEPTED`, usando `PATCH /deliveries/:id/status` com `{ "status": "CANCELLED" }`.

Perfil

A tela de perfil exibe dados do cliente autenticado e oferece logout com confirmação.

Área do Entregador

A tela inicial do entregador exibe os dados do usuário autenticado e uma indicação de que o fluxo operacional completo será implementado na Sprint 4.

3. Atualização Assíncrona

A atualização assíncrona foi implementada por polling, alternativa permitida no enunciado da Sprint 3. A lista de entregas e a tela de detalhe consultam o backend automaticamente a cada 15

segundos, permitindo que mudanças feitas pelo entregador sejam refletidas sem ação manual do cliente.

Na lista, o polling chama `GET /deliveries` enquanto o usuário está autenticado. No detalhe, o polling chama `GET /deliveries/:id` enquanto a entrega ainda não está em estado final. O usuário também pode atualizar manualmente com pull-to-refresh ou pelo botão de atualização, mas essa ação manual não é necessária para o app refletir mudanças feitas no servidor.

4. Arquitetura

```
mobile_app/lib/  
├─ config/           # URL base da API  
├─ controllers/      # estado da sessão e entregas  
├─ models/           # User, AuthSession e Delivery  
├─ screens/          # Login, cliente, entregador, detalhe, criação e perfil  
├─ services/         # cliente HTTP e serviços REST  
├─ theme/            # tema visual do app  
├─ utils/            # helpers de formatação  
└─ widgets/          # componentes reutilizáveis
```

Responsabilidades principais:

- `ApiConfig` define a URL base por `--dart-define=QUICKDELIVERY_API_URL`.
- `ApiClient` centraliza chamadas HTTP, JSON, token Bearer e tratamento de erro.
- `AuthService` e `DeliveriesService` isolam os endpoints REST.
- `AppController` mantém sessão, token, lista de entregas, loading e erros.
- `screens` implementam navegação e telas do cliente/entregador.
- `models` convertem os payloads JSON da API para objetos Dart.

Fluxo principal:

```
Login -> Minhas Entregas -> Nova Entrega -> Detalhes da Entrega  
                                -> Detalhes da Entrega  
                                -> Perfil -> Logout
```

5. Execução

As instruções completas de execução estão no `README.md` da raiz do projeto e no `mobile_app/README.md`. Esses arquivos descrevem como subir o backend, iniciar o emulador Android, configurar `QUICKDELIVERY_API_URL` e rodar o app.

Credenciais seedadas para teste:

Perfil	Email	Senha
Cliente	customer1@example.com	password123
Entregador	deliveryman1@example.com	password123

6. Aderência à Sprint 3

Requisito do enunciado	Implementação no projeto
App Flutter funcional para o cliente	Código em <code>mobile_app</code> , validado em emulador Android.
Mínimo de 3 telas	Login, Minhas Entregas, Nova Entrega, Detalhes da Entrega e Perfil.
Integração com backend REST	Consome <code>POST /auth/login</code> , <code>GET /deliveries</code> , <code>POST /deliveries</code> , <code>GET /deliveries/:id</code> e <code>PATCH /deliveries/:id/status</code> .
Atualização assíncrona de estado	Polling automático a cada 15 segundos na lista e no detalhe.
Arquitetura documentada	Separação em <code>models</code> , <code>services</code> , <code>controllers</code> , <code>screens</code> , <code>widgets</code> , <code>theme</code> , <code>utils</code> e <code>config</code> .
Código-fonte executável	App roda no emulador Android com <code>QUICKDELIVERY_API_URL=http://10.0.2.2:3000</code> .